

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет “Києво-Могилянська академія”
Факультет Інформатики



1615

Лабораторна робота №1
з дисципліни “Алгоритми і структури даних (Java Advanced)”
Тема: Система обліку студентів і викладачів НаУКМА

Студента 1 курсу групи 1
спеціальності F1.
Іженерія програмного
забезпечення
Гордієнка Ігоря
Володимировича
Викладач:
Кирієнко Оксана Валентинівна

м. Київ - 2026

Постановка задачі

Основна мета:

Створити консольну інформаційну систему університету, яка покриває всі теми курсу Advanced Java: ООП, колекції, дженеріки, винятки, логування, функціональне програмування, Stream API, сучасні типи Java, I/O, багатопоточність, мережу, рефлексію та анотації, тестування і роботу з Git.

Конкретні завдання:

1. Проєктування архітектури та моделювання предметної області:

- Розробка об'єктно-орієнтованої моделі консольної системи обліку для НаУКМА.
- Проєктування та реалізація ієрархії класів користувачів: створення базового типу Персона та похідних від нього класів Студент і Викладач із відповідним набором атрибутів.
- Створення структур даних для опису сутностей Університет, Факультет та Кафедра із забезпеченням зв'язків між ними (посилання на викладачів як деканів/завідувачів, прив'язка кафедр до факультетів).

2. Реалізація базового функціоналу управління даними (CRUD-операції):

- Програмування методів для створення, редагування та видалення факультетів і кафедр.
- Реалізація логіки додавання, модифікації та видалення записів про студентів і викладачів у межах конкретних кафедр.
- Розробка функціоналу для переведення студентів між академічними групами чи кафедрами, а також оновлення курсу навчання.

3. Розробка системи пошуку, сортування та генерування звітів:

- Створення алгоритмів пошуку студентів і викладачів за заданими критеріями (ПІБ, курс, група).
- Реалізація функцій виведення відформатованих списків:
 - Загального переліку студентів, впорядкованого за курсами.

- Списків складу факультетів (студенти/викладачі), відсортованих за алфавітом.
- Списків складу кафедр з можливістю сортування як за курсами, так і за алфавітом (зокрема з фільтрацією за певним курсом).

4. Впровадження системи авторизації та розмежування доступу:

- Розробка механізму автентифікації користувачів за логіном та паролем.
- Реалізація рольової моделі керування доступом (Role-Based Access Control) з трьома рівнями:
 - **Користувач:** обмежений доступ (лише функції пошуку та перегляду звітів).
 - **Менеджер:** розширений доступ (можливість управління даними університету, без доступу до налаштувань безпеки).
 - **Адміністратор:** повний доступ, включно зі створенням, редагуванням та блокуванням облікових записів і ролей.

5. Створення користувацького інтерфейсу:

- Розробка інтерактивного консольного меню для спілкування системи з користувачем.
- Забезпечення динамічного відображення пунктів меню залежно від поточної ролі авторизованого користувача.

Розподіл задач

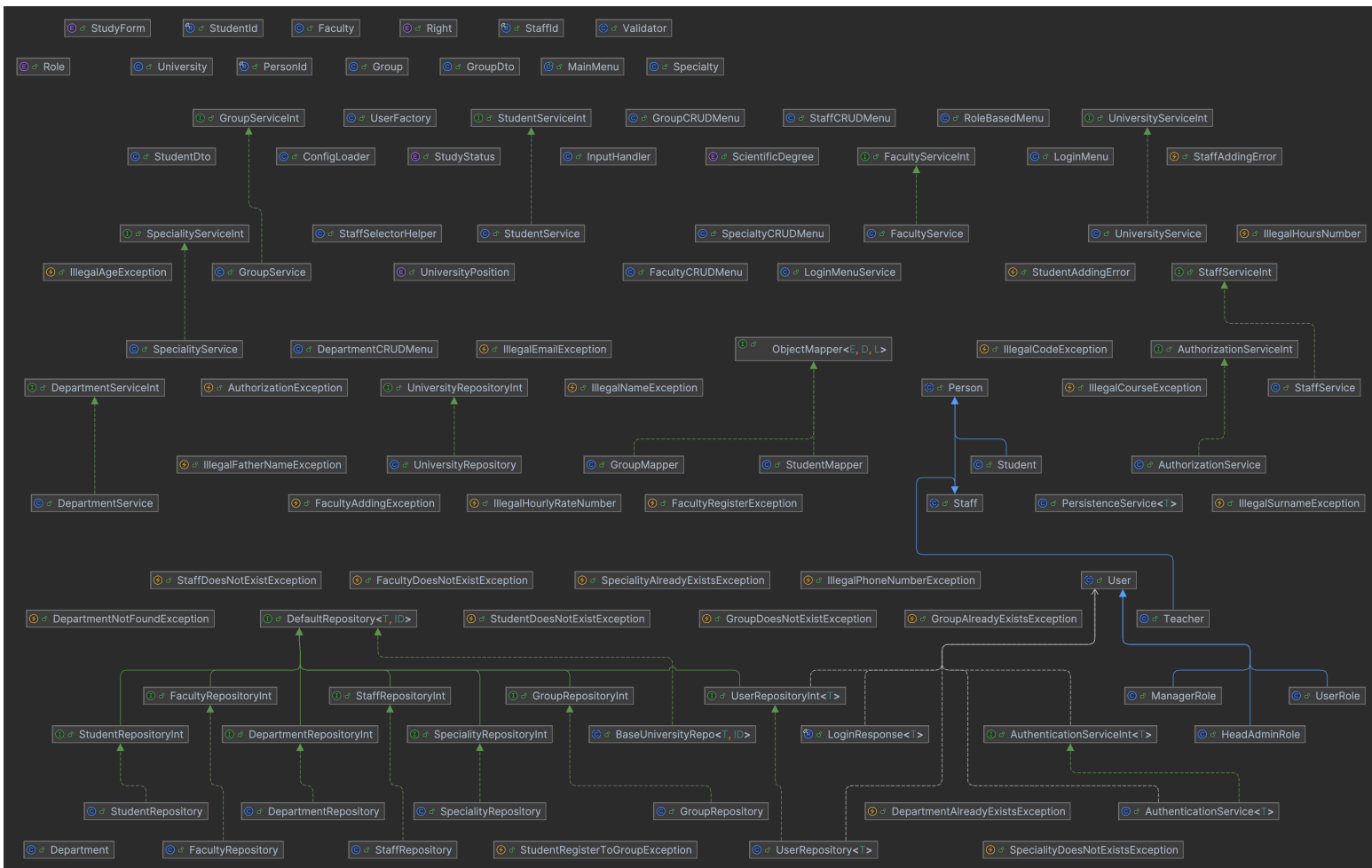
Проект виконувався однією особою, що потребувало взяття на себе усі аспекти розробки.

Гордієнко Ігор Володимирович:

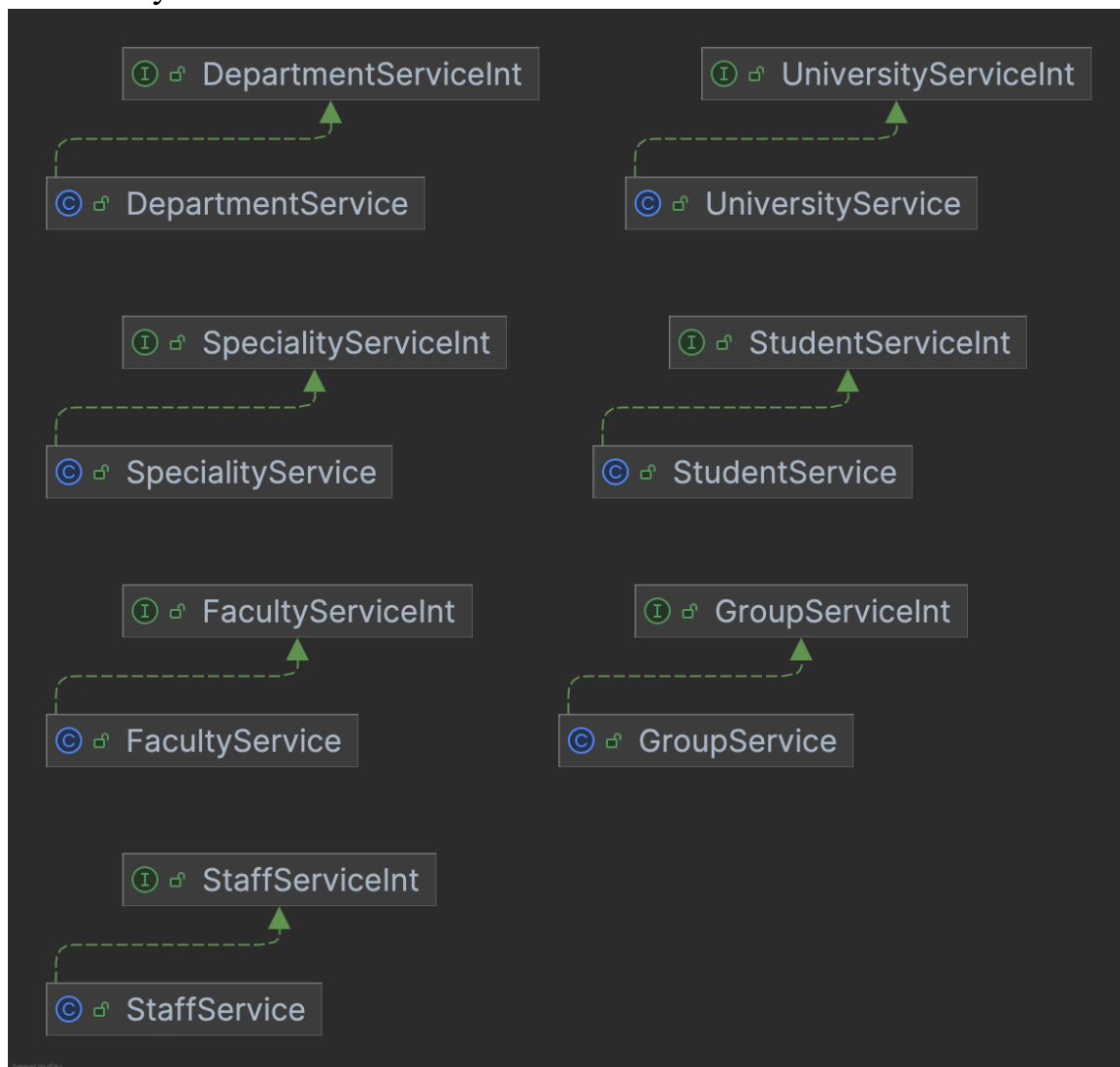
- Проектування архітектури системи
- Розробка базових класів і структури даних
- Реалізація логіки управління користувачами
- Розробка системи оцінювання і звітування
- Тестування та налагодження
- Документування проекту

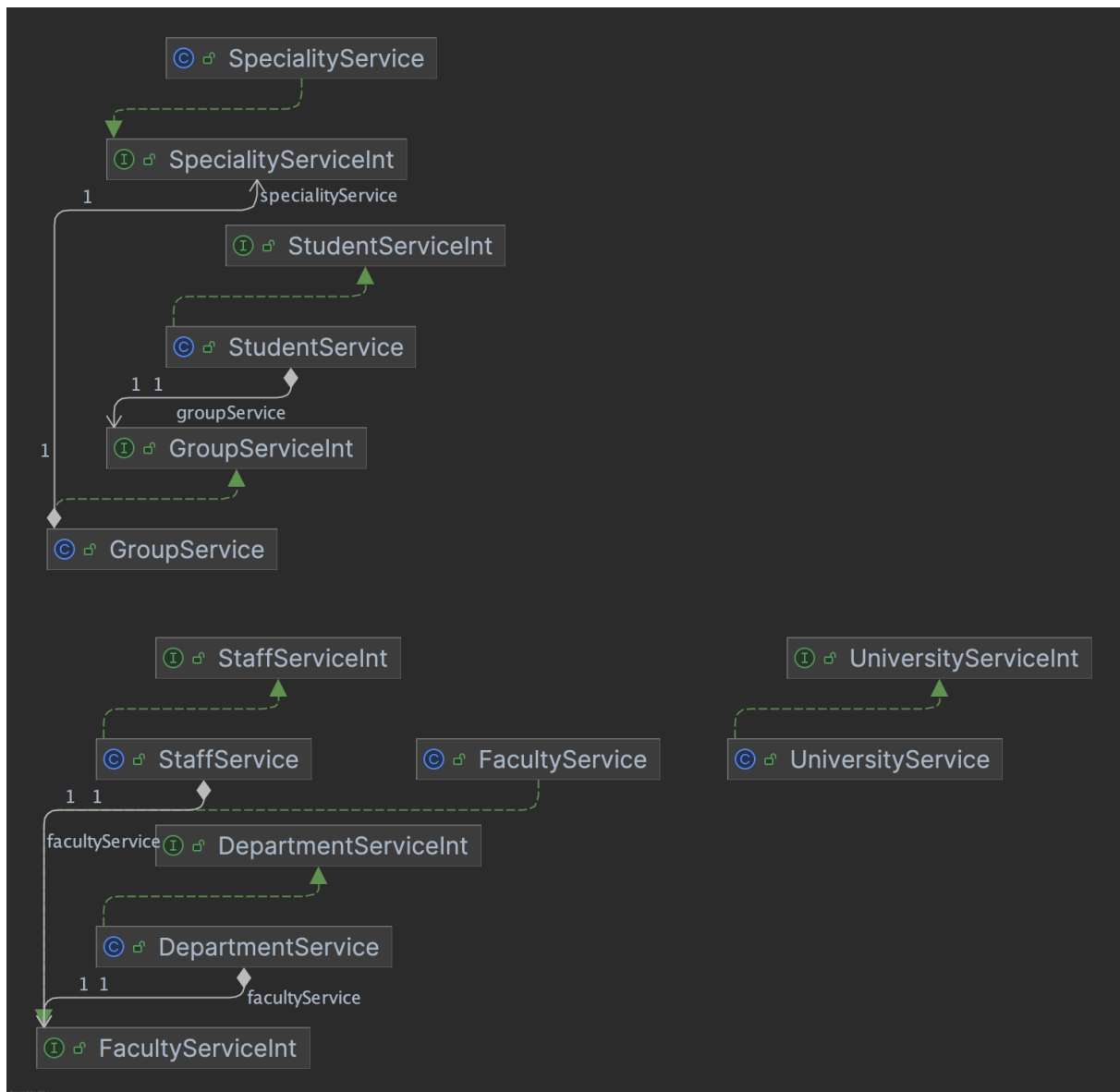
Структура програми

Уся програма:

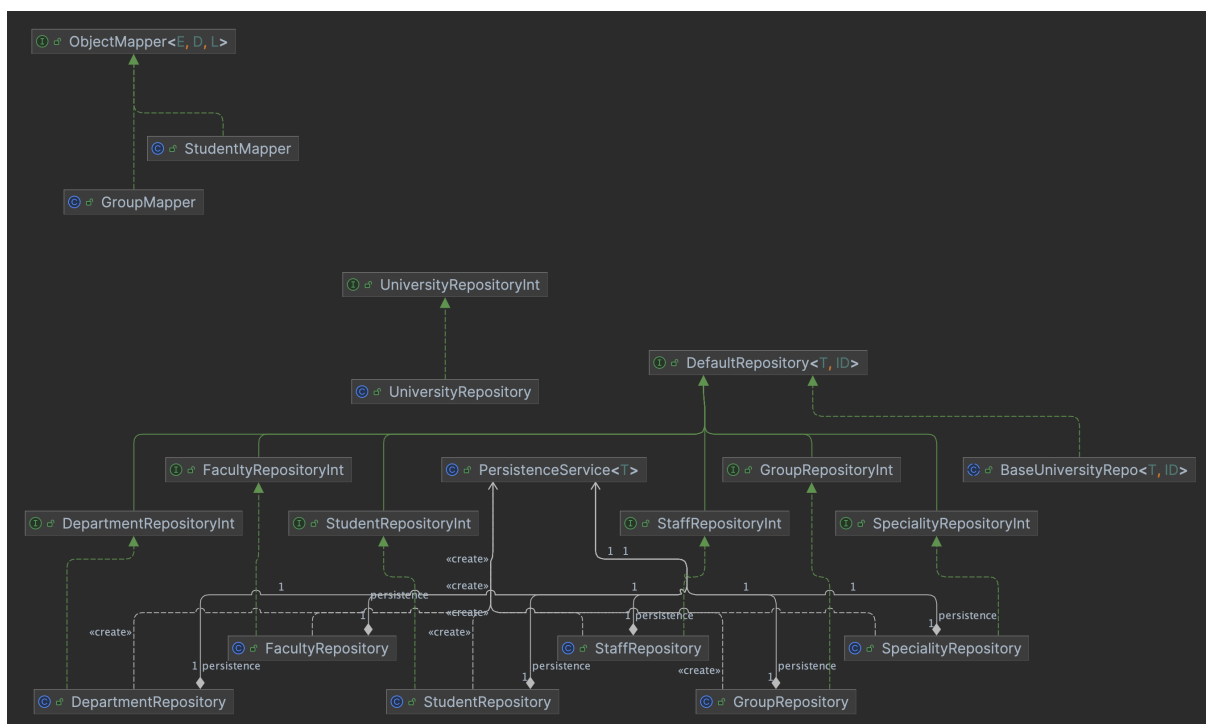
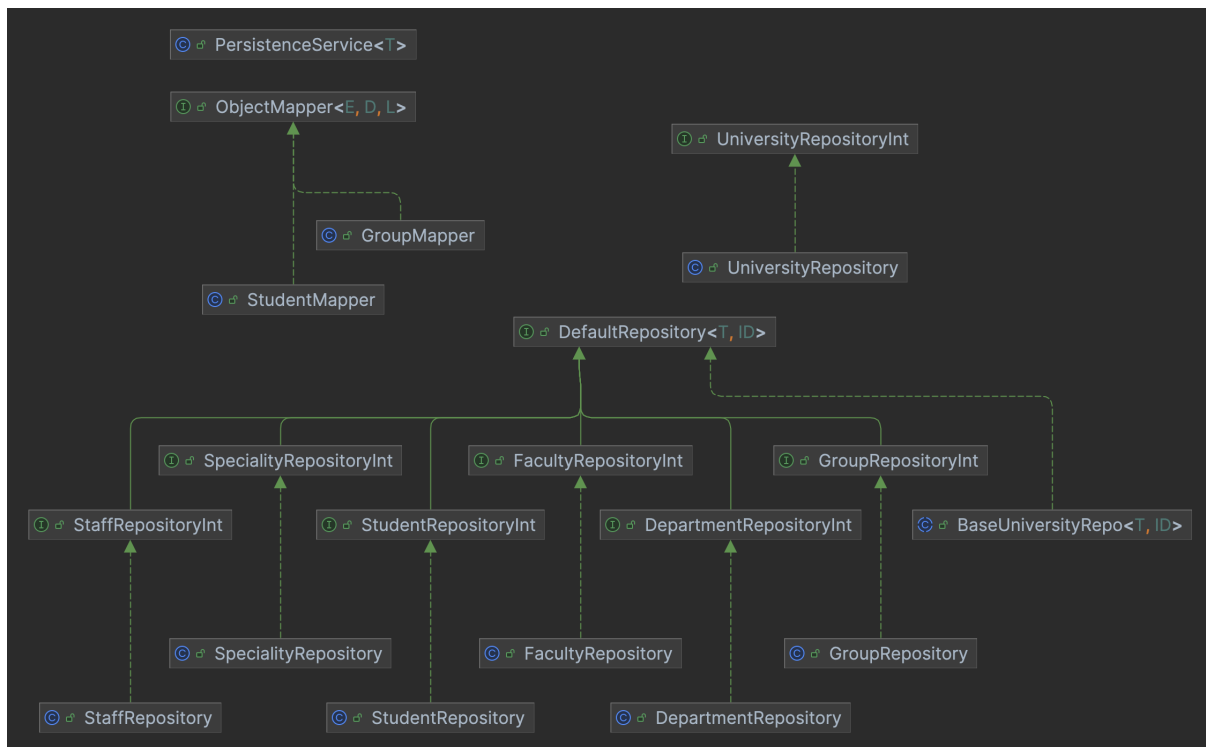


Service layer:

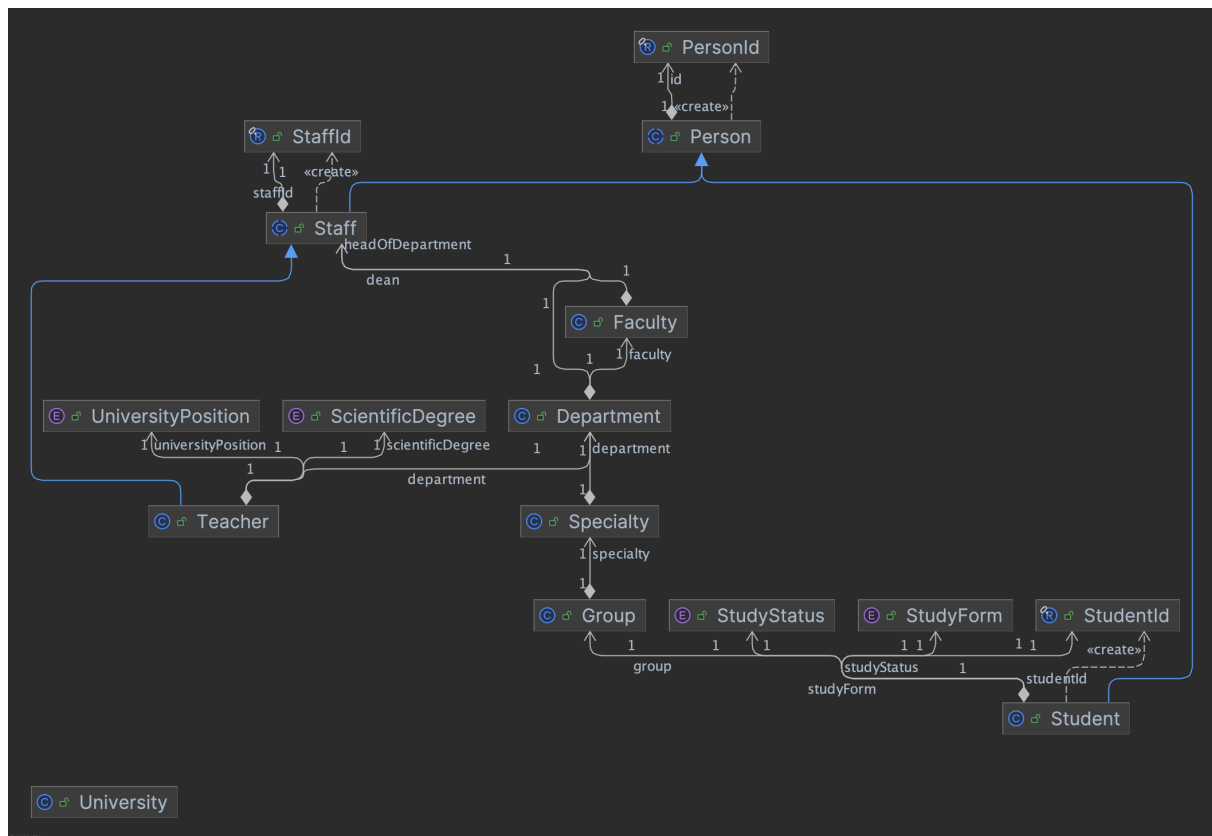




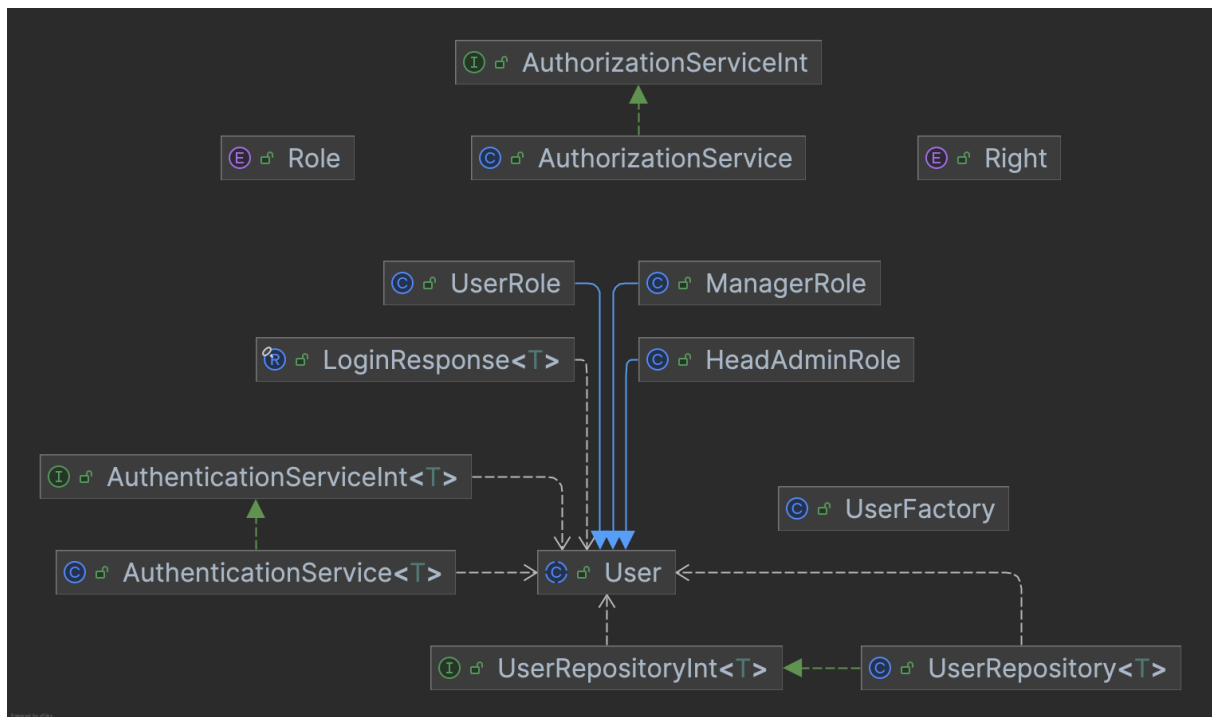
Repository layer:



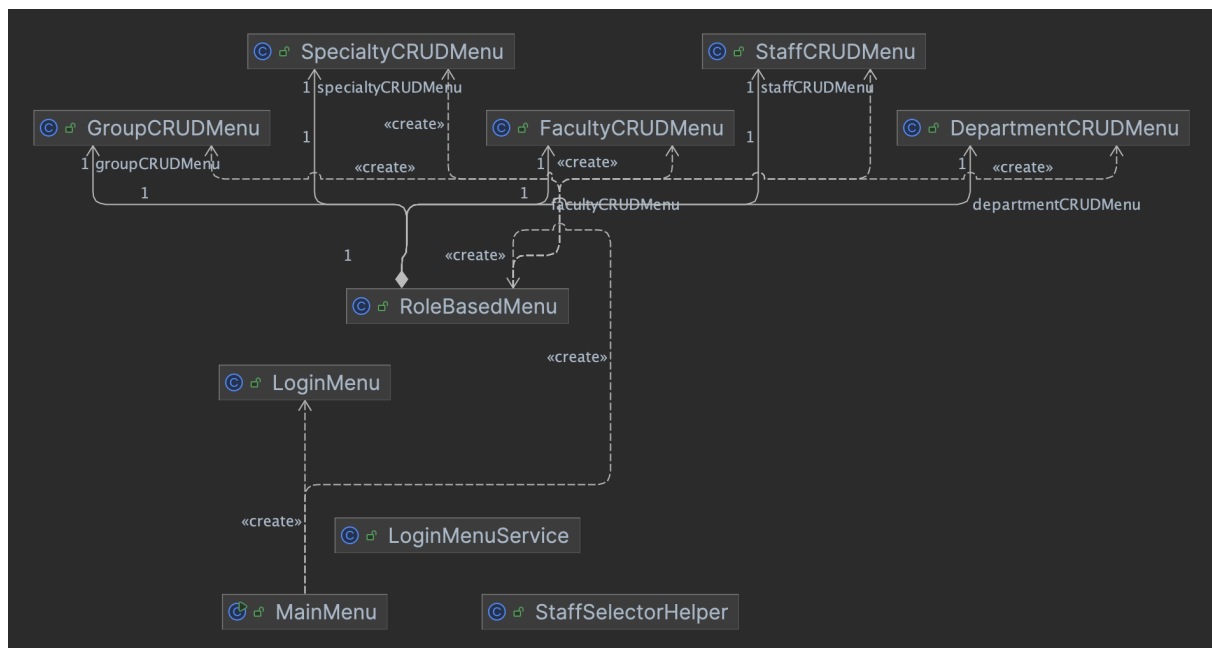
Domain layer:



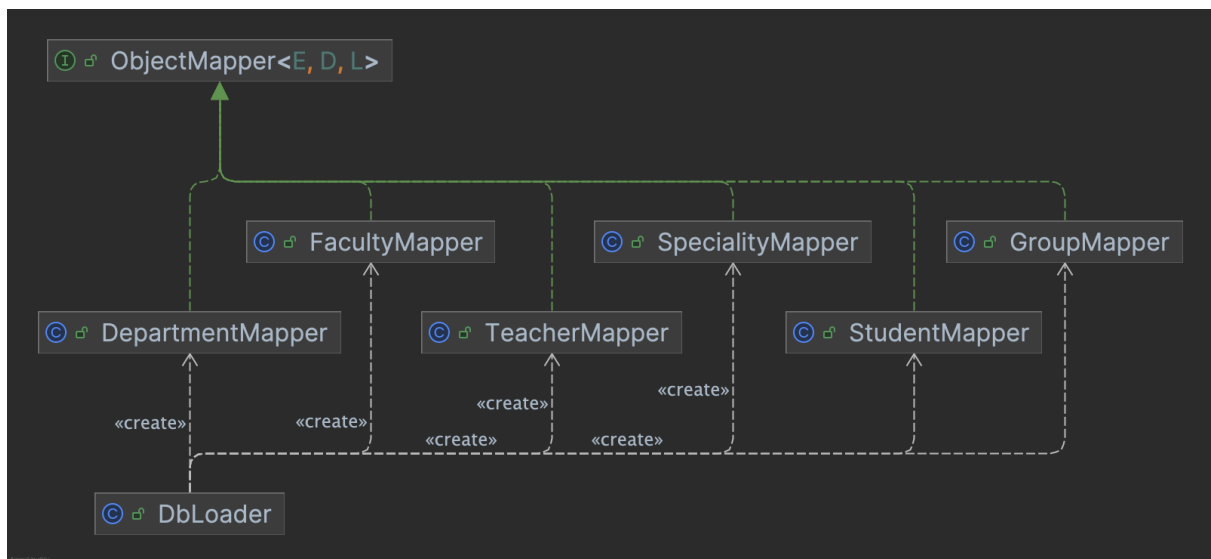
Auth layer:



UI layer:



Mapping:



Опис можливостей

У системі обліку університету реалізовано повний набір функціональних можливостей відповідно до технічного завдання. Система побудована як консольний застосунок мовою Java з підтримкою серіалізації даних у формат JSON за допомогою бібліотеки Jackson та розмежуванням прав доступу на основі ролей.

Модель даних

Система підтримує зберігання та обробку таких сутностей:

University

- повна та скорочена назва;
- місто та адреса.

Faculty

- унікальний код та назва;
- декан — посилання на запис викладача;
- розташування

Department

- унікальний код та назва;

- належність до факультету — посилання на запис факультету;
- завідувач — посилання на запис викладача;
- кабінет або місце розташування.

Specialty

- назва
- унікальний тег
- належність до кафедри -посилання на запис кафедри

Group

- назва
- належність до спеціальності -посилання на запис спеціальності

Person (базовий тип)

- унікальний ідентифікатор (PersonId);
- прізвище, ім'я та по батькові (три окремі поля);
- дата народження, електронна адреса, номер телефону.

Student (нащадок персони Person)

- унікальний ідентифікатор (StudentId);
- курс навчання (1–6), група;
- форма навчання: бюджет або контракт;
- поточний статус: навчається, академічна відпустка, відрахований. очікується.

Staff (нащадок Person)

- унікальний ідентифікатор (StaffId);

Teacher (нащадок Staff)

- посада, науковий ступінь;
- дата прийняття на роботу;
- ставка, навантаження.

Операції керування даними

Система надає повний набір операцій CRUD для всіх основних сутностей:

Управління факультетами:

- створення нового факультету з вказанням усіх реквізитів;
- редагування наявного запису факультету;
- видалення факультету з системи.

Управління кафедрами:

- створення кафедри у межах обраного факультету;
- редагування та видалення запису кафедри.

Управління спеціальностями:

- створення спеціальності у межах обраної кафедри;
- редагування та видалення запису спеціальності.

Управління групами:

- створення групи у межах обраної спеціальності;
- редагування та видалення запису спеціальності.

Управління персоналом і студентами:

- додавання, редагування та видалення студентів у межах кафедри;
- додавання, редагування та видалення викладачів у межах кафедри;
- переведення студента між групами або кафедрами;
- зміна курсу навчання студента.

Пошук та формування звітів

Система підтримує різноманітні сценарії пошуку та виведення інформації:

Пошук:

- пошук студентів або викладачів за прізвищем;
- пошук студента або викладача за специфічним id;
- пошук студента або викладача за специфічним id;
- пошук студента або викладача за ПІБом;

- пошук студентів за курсом або номером групи.

Звіти по університету:

- усі студенти, впорядковані за курсами.

Звіти по факультету:

- усі студенти або викладачі факультету в алфавітному порядку.

Звіти по кафедрі:

- усі студенти кафедри, впорядковані за курсами;
- усі студенти або викладачі кафедри в алфавітному порядку;
- студенти кафедри вказаного курсу — у загальному вигляді та в алфавітному порядку.

Авторизація та розмежування прав доступу

Система підтримує авторизацію за логіном і паролем. Доступ до функцій залежить від призначеної ролі:

Користувач:

- доступ лише до пошуку та перегляду звітів;
- будь-які операції зміни даних заблоковано.

Менеджер:

- повний доступ до CRUD-операцій над факультетами, кафедрами, студентами та викладачами;
- управління обліковими записами користувачів недоступне.

Адміністратор:

- усі права менеджера;
- створення та редагування облікових записів користувачів;
- призначення ролей та блокування облікових записів.

4.5. Серіалізація та зберігання даних

Усі дані системи зберігаються у файлах формату JSON. Серіалізацію та десеріалізацію об'єктів виконує бібліотека Jackson. Такий підхід забезпечує зручне читання файлів даних, незалежність від СУБД та легке

перенесення даних між середовищами. Стан системи автоматично зберігається після кожної операції зміни даних і відновлюється при наступному запуску застосунку.

4.6. Консольний інтерфейс

Взаємодія з системою відбувається через текстове меню. Навігація здійснюється введенням номера пункту меню. Меню адаптується залежно від ролі авторизованого користувача: пункти, недоступні для поточної ролі, не відображаються. Після кожної операції виводиться повідомлення про успішне виконання або опис помилки.

Опис рішень та проблем

Патерн Factory та узагальнений репозиторій

Для створення об'єктів користувачів застосовано патерн Factory: клас UserFactory визначає конкретний тип (HeadAdminRole, ManagerRole, UserRole) на основі рядкового ідентифікатора, що дозволяє розширювати перелік ролей без змін у коді репозиторію. Сховище реалізовано як узагальнений тип UserRepository<T extends User> з інтерфейсом UserRepositoryInt<T> — це забезпечує залежність від абстракції та спрощує підміну реалізації.

Серіалізація даних: Jackson та JSON

Основні сутності системи (факультети, кафедри, спеціальності, групи студенти, викладачі) серіалізуються у формат JSON за допомогою бібліотеки Jackson. Такий підхід усуває необхідність у СУБД, забезпечує зручне читання файлів даних та легке перенесення між середовищами. Головна складність полягала у серіалізації ієрархій класів: оскільки Student і Teacher успадковують базовий тип Person, Jackson потребує анотацій @JsonTypeInfo та @JsonSubTypes для коректного відновлення конкретного типу при десеріалізації. Без цих анотацій ObjectMapper не може визначити, який підклас інстанціювати. Також потрібно було явно забезпечити наявність конструктора без аргументів, щоб Jackson міг створювати об'єкти під час зчитування.

Розділення сховищ для захисту облікового запису адміністратора

Обліковий запис головного адміністратора зберігається в окремому файлі head_admin_config.csv, який завантажується при старті, але ніколи не перезаписується операціями CRUD. Метод saveAllToFile() явно виключає екземпляри HeadAdminRole із запису, що унеможливорює випадкове або навмисне видалення адміністраторського запису через інтерфейс системи.

Небезпечне приведення типів

У UserRepository присутнє явне приведення при створенні об'єктів через фабрику. Компілятор видає попередження unchecked cast, оскільки узагальнений параметр T стирається під час виконання. Ризик реального ClassCastException відсутній, бо типи контролюються в одному місці, однак повноцінним рішенням було б передавання Class<T> у конструктор репозиторію для явної перевірки типу.

Мапінг об'єктів і їх лінкування

У якийсь момент я натрапив на досить очевидний баг пов'язаний з тим як Jackson зберігає вкладені об'єкти. Тому що бібліотека коли бачить вкладений об'єкт просто вписує його повністю в поле, включно з вже його вкладеними об'єктами, якщо назначити викладача головою кафедри і призначити викладачу посилання ту саму кафедру, персистенція впаде з переповненням стеку.

Як я вирішив цю проблему. Було реалізовано класи мапери, які вміють перетворювати дані з інстанс в дто і з дто в інстанс. Але тут постала інша проблема, що треба лінкувати тепер об'єкти, тому що дто зберігає тільки одне поле по якому можна ідентифікувати потрібний зв'язаний об'єкт. Було реалізовано клас DbLoader який приймає репозиторії, отримує списки не злінкованих об'єктів та дто і у певному порядку їх зв'язує між собою.

Код Програми

[Посилання на GitHub](#)

Висновки

У ході виконання лабораторної роботи було розроблено консольну інформаційну систему обліку студентів і викладачів мовою Java. Система реалізує повний набір CRUD-операцій для управління факультетами, кафедрами, спеціальностями, групами, студентами та викладачами. Розроблено рольову модель доступу (користувач, менеджер, адміністратор) з авторизацією за логіном і паролем. Дані зберігаються у форматі JSON за допомогою бібліотеки Jackson, що забезпечує незалежність від СУБД.

Під час розробки було застосовано патерн Factory для керування ролями, узагальнені репозиторії для абстрагування логіки зберігання, а також класи-маппери для вирішення проблеми циклічних посилань при серіалізації.

Результатом є повнофункціональний застосунок, що охоплює основні теми курсу Advanced Java: ООП, дженеріки, колекції, виключення, серіалізацію, розмежування прав доступу, багатопоточність, анотації, логування, Stream API.